

Trustworthy AI Assignment 4

Hanyoung An (2021440073)

Abstract—This report studies neural network verification with alpha-beta-CROWN. First, I explore the built-in model and configuration structure of the verifier. Then, I run alpha-beta-CROWN on an external ONNX model, the MNIST wide MLP from Assignment 3, and compare the results with the previous Marabou baseline. The experiment verifies local ℓ_∞ robustness for pixel perturbations of 1 and 5, but finds a counterexample for perturbation 10.

I. INTRODUCTION

Neural network verification checks whether a trained model satisfies a formal property over an input region. In adversarial robustness verification, the property is usually that all inputs inside an ℓ_∞ ball around a sample keep the same predicted label. Assignment 3 used Marabou, an SMT-style neural network verifier, to solve this type of property. In this assignment, I use alpha-beta-CROWN, which combines bound propagation, linear relaxation, alpha/beta optimization, and branch-and-bound (BaB) [1].

The goals are twofold. First, I inspect the alpha-beta-CROWN repository to understand its built-in models and YAML-based workflow. Second, I verify an external model that is not included in the repository’s built-in model directory. For the external experiment, I reuse the MNIST wide MLP from Assignment 3 so that the alpha-beta-CROWN result can be compared directly with the existing Marabou result [2].

II. EXPLORATION REPORT

I inspected the built-in models directory and the example configuration directory of alpha-beta-CROWN. The models directory contains 37 checkpoint or specification-related files: 24 PyTorch checkpoint files, 8 model files, 4 pickle specification files, and 1 additional PyTorch file. Most of them are PyTorch-style checkpoints used by predefined model constructors. The examples cover CIFAR-10 ResNets, ERAN MNIST/CIFAR networks, SDP CNNs, OVAL CIFAR networks, Marabou comparison models, non-ReLU sigmoid networks, and custom toy examples.

The configuration directory contains 262 YAML files. They are grouped into tutorial examples, beta-CROWN examples, GCP-CROWN examples, BICCOS experiments, BaB-Attack examples, and VNN-COMP benchmark configurations. A typical YAML file contains sections for the general run setting, model, data, specification, attack, solver, and BaB options. This is different from Marabou, where the model and constraints are usually assembled through Python API calls or query construction. Table I summarizes the main workflow difference.

TABLE I: Verifier workflow comparison

Item	Marabou	alpha-beta-CROWN
Approach	SMT solving	Bounds + BaB
Model input	API / ONNX query	YAML model path
Property	Python constraints	YAML / VNNLIB
Result	SAT/UNSAT/TIMEOUT	verified/falsified/timeout

III. IMPLEMENTATION AND RESULTS

A. External Model and Property

The external model is the Assignment 3 MNIST wide MLP exported to ONNX. It is stored under the local models directory. The architecture is 784-128-10: a flattened MNIST input, one hidden ReLU layer of width 128, and 10 output logits. The model input is raw MNIST pixel values in $[0, 255]$, and the pixel normalization is folded into the first affine layer. The selected sample is MNIST test sample index 0, whose true label and predicted label are both 7. This is the same model, sample, and robustness setup used in my Assignment 3 Marabou experiment, so the later comparison is a direct verifier-to-verifier comparison rather than a comparison across different datasets.

The verification property is local ℓ_∞ robustness. For epsilon values 1, 5, and 10 in raw pixel scale, I check whether every perturbed input x' satisfying

$$\|x' - x\|_\infty \leq \epsilon \quad (1)$$

keeps class 7 as the top logit. I generated one VNNLIB file for each epsilon. The VNNLIB encodes the unsafe region:

$$\exists t \neq 7 \text{ s.t. } Y_t \geq Y_7. \quad (2)$$

Therefore, an unsatisfiable result means that the unsafe region is empty and the robustness property is verified, while a satisfiable result means that an adversarial counterexample exists.

B. Implementation

The reproducible runner is the provided Python script, test.py. It executes three YAML configurations, one for each epsilon value. Each configuration uses CUDA, complete BaB verification, 100 PGD steps with 30 restarts, alpha-CROWN iteration 50, beta-CROWN iteration 20, batch size 4096, and KFSB branching. The preparation scripts copy the Assignment 3 assets and generate the VNNLIB specifications. The raw logs are saved under the results directory, and the combined comparison is saved as a CSV summary. The implementation deliverables are the runner, the three YAML configuration files, the external ONNX model, the MNIST sample, the generated VNNLIB specifications, and the recorded result logs.

C. Results and Marabou Comparison

Table II shows the final observed results. For epsilon 1, alpha-beta-CROWN verified the property with initial CROWN bounds. For epsilon 5, PGD did not find a violation, and alpha-CROWN optimization verified the remaining constraint. For epsilon 10, PGD quickly found a counterexample, so the property was falsified.

TABLE II: Verification results

ϵ	alpha-beta-CROWN	Time	Marabou baseline
1.0	verified (unsat)	7.43s	UNSAT, 1.66s
5.0	verified (unsat)	7.56s	UNSAT, 9.43s
10.0	falsified (sat)	6.81s	SAT+TIMEOUT, 145.86s

IV. COMPARISON AND DISCUSSION

The two verifiers agree on the robustness trend. For epsilons 1 and 5, both tools indicate that the model is locally robust. For epsilon 10, both indicate that robustness no longer holds. The main difference is scalability and termination behavior. Marabou was faster for the smallest epsilon, but for epsilon 10 it had target-level timeouts and took 145.86 seconds. In contrast, alpha-beta-CROWN found a counterexample in 6.81 seconds through its PGD pre-attack. This experiment shows that alpha-beta-CROWN’s combination of attack-based falsification and GPU-based bound computation can be more practical when the property is false or the perturbation radius is larger. In terms of ease of use, alpha-beta-CROWN was convenient for repeated experiments because the model, property, and solver parameters were all kept in YAML files, while Marabou gave more direct control through Python-level constraints. Feature-wise, alpha-beta-CROWN’s ONNX/VNNLIB workflow and GPU BaB support were well matched to this robustness experiment.

V. CONCLUSION

This assignment confirmed that alpha-beta-CROWN uses a more configuration-driven workflow than Marabou. The repository’s built-in examples are organized around YAML files that specify the model, data, property, solver options, and BaB settings. For the external ONNX MNIST wide MLP, alpha-beta-CROWN verified robustness for epsilons 1 and 5 and falsified robustness for epsilon 10. The results are consistent with the Marabou baseline, while alpha-beta-CROWN was especially faster in the falsified epsilon 10 case. A practical lesson from the implementation is that VNNLIB direction must be handled carefully: the verifier proves robustness by showing that the asserted unsafe region is unsatisfiable.

REFERENCES

- [1] S. Wang, H. Zhang, K. Xu, X. Lin, S. Jana, C.-J. Hsieh, and J. Z. Kolter, “Beta-CROWN: Efficient bound propagation with per-neuron split constraints for neural network robustness verification,” *Advances in Neural Information Processing Systems*, vol. 34, pp. 29 909–29 921, 2021.
- [2] G. Katz, D. A. Huang, D. Ibeling, K. Julian, C. Lazarus, R. Lim, P. Shah, S. Thakoor, H. Wu, A. Zeljić, D. L. Dill, M. J. Kochenderfer, and C. Barrett, “The Marabou framework for verification and analysis of deep neural networks,” in *Proceedings of the 31st International Conference on Computer Aided Verification*, ser. Lecture Notes in Computer Science, I. Dillig and S. Tasiran, Eds., vol. 11561. Springer International Publishing, Jul. 2019, pp. 443–452.